

Tytuł: **Guess Who**

Autorzy: **Miłosz Matras (MM), Karolina Matyszkowicz (KM)**

Ostatnia modyfikacja: **15.06.2026**

1.	Repozytorium git.....	1
2.	Wstęp.....	1
3.	Specyfikacja.....	2
3.1.	Opis ogólny algorytmu	2
3.2.	Tabela zdarzeń.....	2
4.	Architektura	3
4.1.	Moduł: top.....	3
4.1.1.	Schemat blokowy.....	4
4.1.2.	Porty.....	4
a)	porty zewnętrzne modułu top_basys3.....	3
b)	porty modułu top_vga.....	3
4.1.3.	Interfejsy.....	5
a)	interfejsy logiczne i wewnętrzne top_vga.....	3
4.2.	Rozprowadzenie sygnału zegara	5
5.	Implementacja.....	6
5.1.	Lista zignorowanych ostrzeżeń Vivado.	6
5.2.	Wykorzystanie zasobów.....	6
5.3.	Marginesy czasowe.....	6
6.	Konfiguracja sprzętu	6
7.	Film.	7

1. **Repozytorium git**

Adres repozytorium GITa:

https://github.com/MMS5-lang/PROJECT_GUESSWHO

Repozytorium zawiera źródła RTL, testbenche, skrypty do symulacji i generowania bitstreamu, dokumentację projektu oraz katalog results z wygenerowanym plikiem top_basys3.bit.

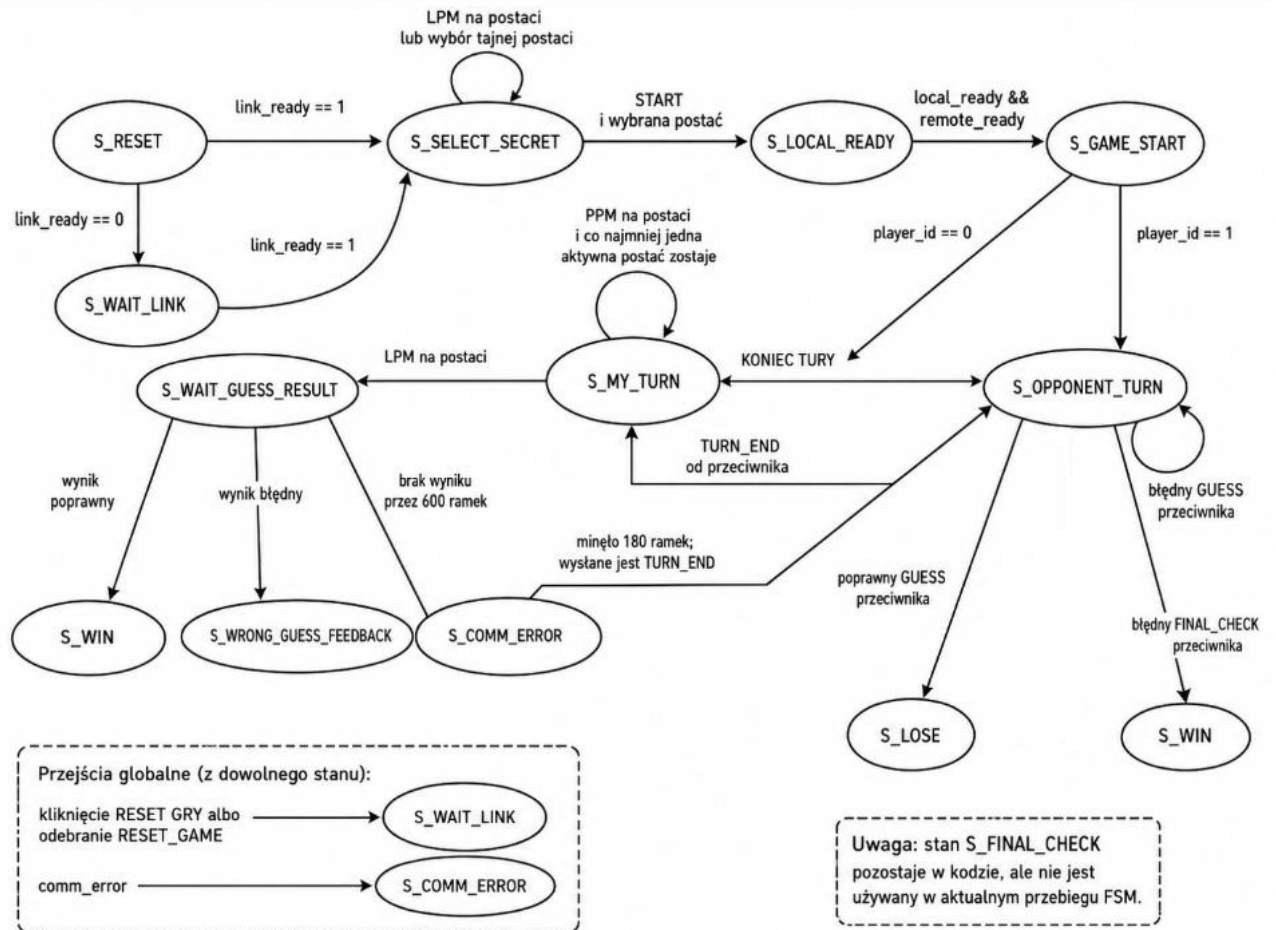
2. **Wstęp**

Projekt realizuje sprzętową wersję gry Guess Who dla dwóch płytek Digilent Basys 3. Każdy gracz korzysta z własnej płytki, monitora VGA 1024 x 768 oraz myszy PS/2. Płytki komunikują się przez UART wyprowadzony na złącze PMOD JA. FPGA odpowiada za wybór tajnej postaci, prowadzenie tur, lokalne eliminowanie kandydatów,

zgadywanie postaci przeciwnika, obsługę wyniku oraz reset gry. Rozmowa i pytania między graczami odbywają się poza układem, tak jak w klasycznej grze.

3. Specyfikacja

3.1. Opis ogólny algorytmu



3.2. Tabela zdarzeń

Tabela obejmuje zdarzenia obsługiwane w game_core.sv oraz zdarzenia pakietowe generowane przez pmod_comm_controller.sv.

Zdarzenie	Źródło w kodzie	Stan / warunek	Reakcja systemu
Reset sprzętowy	!rst_n	dowolny stan	Rejestry gry przechodzą do wartości początkowych, a FSM startuje od S_RESET.
Zestawiony link	link_ready	S_RESET albo S_WAIT_LINK	Przejście do S_SELECT_SECRET, czyli wyboru tajnej postaci.
Brak linku po resecie	link_ready = 0	S_RESET	Przejście do S_WAIT_LINK i oczekiwanie na poprawny pakiet UART.
LPM na postaci	char_left_click + char_id	S_SELECT_SECRET, poprawne ID	Zapis selected_id oraz ustawienie has_secret.
START po wyborze	start_click	S_SELECT_SECRET i has_secret = 1	Zapis local_secret_id, ustawienie local_ready, impuls send_ready i przejście do S_LOCAL_READY.
READY od przeciwnika	opponent_ready	dowolny stan poza resetem/błędem	Ustawienie remote_ready.
Obaj gracze gotowi	local_ready && remote_ready	S_LOCAL_READY	Przejście do S_GAME_START.
Ustalenie pierwszej tury	player_id	S_GAME_START	player_id=0 przechodzi do S_MY_TURN, player_id=1 do S_OPPONENT_TURN.
LPM na postaci	char_left_click + char_id	S_MY_TURN, poprawne ID	Zapis last_guess_id, impuls send_guess i przejście do S_WAIT_GUESS_RESULT.

PPM na postaci	char_right_click + char_id	S_MY_TURN, poprawne ID	Przełączenie bitu eliminated_mask, ale tylko jeśli po operacji zostaje co najmniej jedna aktywna postać.
KONIEC TURU	start_click	S_MY_TURN	Impuls send_turn_end i przejście do S_OPPONENT_TURN.
Wynik GUESS poprawny	guess_result_valid + guess_result_correct	S_WAIT_GUESS_RESULT	Przejście do S_WIN.
Wynik GUESS błędny	guess_result_valid + !guess_result_correct	S_WAIT_GUESS_RESULT	Dodanie last_guess_id do eliminated_mask, wyzerowanie liczników i przejście do S_WRONG_GUESS_FEEDBACK.
Timeout wyniku GUESS	frame_tick + result_wait_cnt	S_WAIT_GUESS_RESULT	Po 600 ramkach bez wyniku przejście do S_COMM_ERROR.
Koniec feedbacku błędnego strzału	frame_tick + feedback_cnt	S_WRONG_GUESS_FEEDBACK	Po 180 ramkach impuls send_turn_end i przejście do S_OPPONENT_TURN.
Wynik FINAL_CHECK poprawny	final_result_valid + final_result_correct	S_FINAL_CHECK	Przejście do S_WIN.
Wynik FINAL_CHECK błędny	final_result_valid + !final_result_correct	S_FINAL_CHECK	Przejście do S_LOSE.
Timeout FINAL_CHECK	frame_tick + result_wait_cnt	S_FINAL_CHECK	Po 600 ramkach bez wyniku przejście do S_COMM_ERROR.
GUESS od przeciwnika	opponent_guess + opponent_guess_id	S_OPPONENT_TURN, poprawne ID	Porównanie z local_secret_id, impuls send_result; przy trafieniu lokalna gra przechodzi do S_LOSE.
FINAL_CHECK od przeciwnika	opponent_final_check + ID	S_OPPONENT_TURN, poprawne ID	Porównanie z local_secret_id, impuls send_result; trafienie daje S_LOSE, pomyłka daje S_WIN.
TURN_END od przeciwnika	opponent_turn_end	S_OPPONENT_TURN	Przejście do S_MY_TURN.
RESET GRY lokalny	reset_click	dowolny stan	Wyczyszczenie stanu gry, impuls send_reset_game i powrót do S_WAIT_LINK.
RESET_GAME od przeciwnika	opponent_reset_game	dowolny stan	Wyczyszczenie stanu gry bez odsyłania resetu i powrót do S_WAIT_LINK.
Błąd komunikacji	comm_error	dowolny stan	Przejście do S_COMM_ERROR, wyzerowanie liczników oczekiwania i feedbacku.
Stan końcowy wygranej	S_WIN	brak resetu	FSM pozostaje w S_WIN.
Stan końcowy przegranej	S_LOSE	brak resetu	FSM pozostaje w S_LOSE.
Niepoprawne ID postaci	char_id/opponent_*_id >= CHAR_COUNT	kliknięcie albo pakiet z ID	Zdarzenie jest ignorowane przez logikę gry albo odrzucone w kontrolerze komunikacji.
Pakiet HELLO lub poprawny pakiet UART	PKT_HELLO / valid packet	pmod_comm_controller	Ustawia link_ready i odświeża timeout komunikacji.
Pakiet ACK	PKT_ACK	pmod_comm_controller	Potwierdza pakiet niezawodny i pozwala nadać kolejne zdarzenie.
Timeout ACK / retransmisje	ack_timeout, retry_count	pmod_comm_controller	Ponowienie pakietu; po przekroczeniu MAX_RETRIES ustawienie comm_error.
Timeout braku pakietów	comm_timeout_counter	pmod_comm_controller	Po czasie bez poprawnych pakietów link_ready jest czyszczone i ustawiany jest comm_error.

4. Architektura

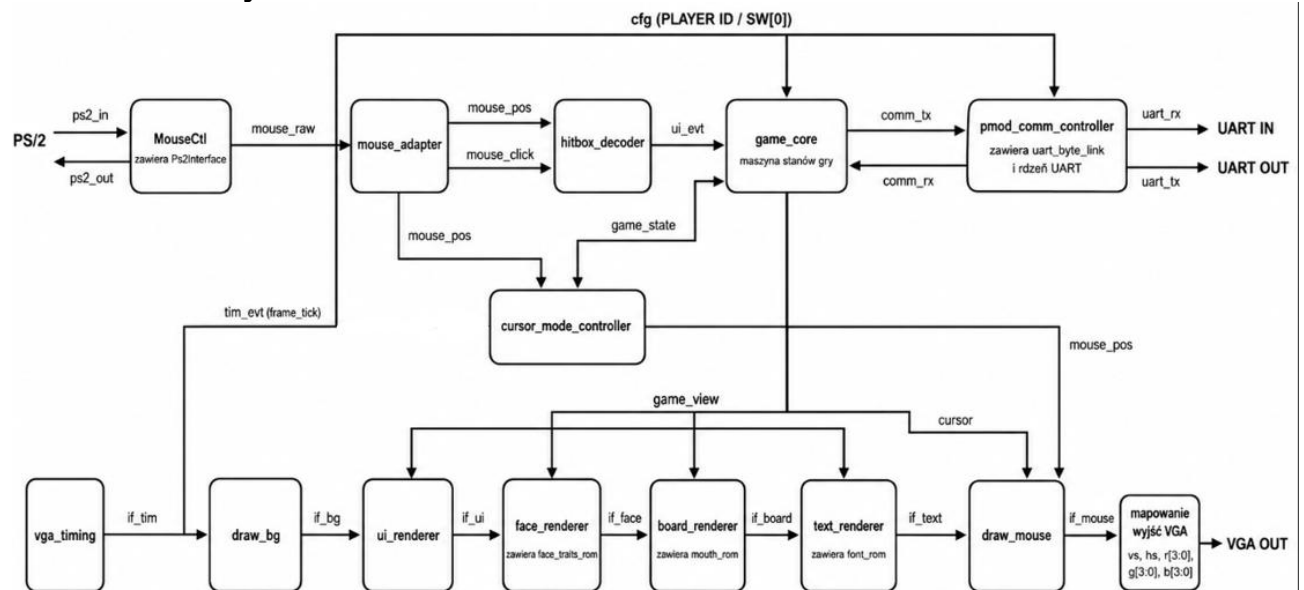
Architektura została podzielona na moduły funkcjonalne odpowiedzialne za logikę gry, komunikację UART, obsługę myszy PS/2, dekodowanie kliknięć oraz potok renderowania VGA. Moduły renderujące są połączone kolejnymi interfejsami vga_if, dzięki czemu każda warstwa obrazu dopisuje swój fragment do sygnału pikselowego.

Topem syntezowalnym dla płytki jest fpga/rtl/top_basys3.sv. Wewnątrz znajduje się IP clk_wiz_0, układy resetu oraz funkcjonalny top gry rtl/top/top_vga.sv.

4.1. Moduł: top

Osoby odpowiedzialne: MM, KM. Moduł top_basys3 łączy zasoby fizycznej płytki Basys 3 z modułem top_vga, generuje zegary 100 MHz i 65 MHz oraz synchronizuje reset w obu domenach zegarowych.

4.1.1. Schemat blokowy



4.1.2. Porty

a) porty zewnętrzne modułu top_basys3

Port top_basys3	Kierunek	Interfejs	Opis
clk	wejście	zegar	Zegar 100 MHz z płytki Basys 3.
btnC	wejście	reset	Fizyczny reset sprzętowy, filtrowany przez debounce.
sw[0]	wejście	cfg	Identyfikator gracza player_id; jedna płytka ma 0, druga 1.
JA3	wejście	uart_rx	Wejście UART RX z drugiej płytki.
PS2Clk	inout	ps2	Dwukierunkowa linia zegara myszy PS/2.
PS2Data	inout	ps2	Dwukierunkowa linia danych myszy PS/2.
vgaRed[3:0]	wyjście	vga	Czterobitowa składowa czerwona RGB444.
vgaGreen[3:0]	wyjście	vga	Czterobitowa składowa zielona RGB444.
vgaBlue[3:0]	wyjście	vga	Czterobitowa składowa niebieska RGB444.
Hsync	wyjście	vga	Synchronizacja pozioma VGA.
Vsync	wyjście	vga	Synchronizacja pionowa VGA.
JA1	wyjście	pclk_debug	Pomocnicze wyjście lustrzanego zegara pikselowego przez ODDR.
JA2	wyjście	uart_tx	Wyjście UART TX do drugiej płytki.

b) porty modułu top_vga

Port top_vga	Kierunek	Interfejs	Opis
clk	wejście	zegar	Zegar 65 MHz dla VGA, logiki gry, renderowania i komunikacji.
clk_100mhz	wejście	zegar	Zegar 100 MHz używany przez kontroler myszy PS/2.
rst_n	wejście	reset	Reset aktywny stanem niskim w domenie 65 MHz.
rst_100mhz_n	wejście	reset	Reset aktywny stanem niskim w domenie 100 MHz.
player_id	wejście	cfg	Identyfikator gracza z SW[0].
pmod_uart_rx	wejście	uart_rx	Odbiór UART z PMOD JA3.
ps2_clk	inout	ps2	Linia zegara PS/2 przekazana do MouseCtl.
ps2_data	inout	ps2	Linia danych PS/2 przekazana do MouseCtl.
pmod_uart_tx	wyjście	uart_tx	Nadawanie UART na PMOD JA2.
r[3:0]	wyjście	vga	Czerwona składowa RGB444 z ostatniego etapu vga_if.

g[3:0]	wyjście	vga	Zielona składowa RGB444 z ostatniego etapu vga_if.
b[3:0]	wyjście	vga	Niebieska składowa RGB444 z ostatniego etapu vga_if.
hs	wyjście	vga	Synchronizacja pozioma VGA.
vs	wyjście	vga	Synchronizacja pionowa VGA.

4.1.3. Interfejsy

a) *interfejsy logiczne i wewnętrzne top_vga*

Interfejs	Kierunek	Składowe	Znaczenie
ps2_in / ps2_out	Mysz PS/2 <-> MouseCtl	ps2_clk, ps2_data	Interfejs dwukierunkowy PS/2 rozbity na kierunek odbioru i sterowania liniami.
mouse_raw	MouseCtl -> mouse_adapter	mouse_xpos_raw, mouse_ypos_raw, mouse_left_raw, mouse_right_raw	Surowe położenie i przyciski z kontrolera PS/2.
mouse_pos	mouse_adapter -> hitbox/cursor/draw_mouse	mouse_x, mouse_y	Pozycja kursora po synchronizacji i ograniczeniu do obszaru ekranu.
mouse_click	mouse_adapter -> hitbox_decoder	left_click_pulse, right_click_pulse	Jednocyfrowe impulsy lewego i prawego kliknięcia.
ui_evt	hitbox_decoder -> game_core	start_click, reset_click, char_left_click, char_right_click, clicked_char_id	Zdarzenia UI wygenerowane na podstawie pozycji kursora i kliknięć.
comm_tx	game_core -> pmod_comm_controller	send_ready, send_turn_end, send_guess, send_guess_id, send_final_check, send_final_check_id, send_result, send_result_correct, send_result_id, send_reset_game	Zdarzenia gry przeznaczone do wysłania przez UART.
comm_rx	pmod_comm_controller -> game_core	link_ready, comm_error, opponent_ready, opponent_turn_end, opponent_guess, opponent_guess_id, opponent_final_check, opponent_final_check_id, opponent_reset_game, guess_result_valid, guess_result_correct, final_result_valid, final_result_correct	Zdarzenia odebrane z drugiej płytki oraz status linku.
game_view	game_core -> renderery	game_state, eliminated_mask, selected_id, last_guess_id, has_secret, local_ready, remote_ready	Stan gry używany do rysowania planszy, panelu, tekstu i trybu kursora.
cursor	cursor_mode_controller -> draw_mouse	cursor_mode	Wybrany wariant kursora: zwykły, hover albo busy.
cfg	top_vga -> wybrane moduły	player_id	Konfiguracja identyfikatora gracza dla logiki, komunikacji i tła.
tim_evt	vga_timing -> game_core	frame_tick	Impuls początku ramki używany do liczników timeoutu i feedbacku.
if_tim	vga_timing -> draw_bg	vga_if: hcount, vcount, hsync, vsync, hblnk, vblnk, rgb	Początek potoku renderowania VGA.
if_bg	draw_bg -> ui_renderer	vga_if	Obraz po narysowaniu tła i siatki planszy.
if_ui	ui_renderer -> face_renderer	vga_if	Obraz po panelu bocznym i przyciskach.
if_face	face_renderer -> board_renderer	vga_if	Obraz po narysowaniu postaci.
if_board	board_renderer -> text_renderer	vga_if	Obraz po nakładkach stanu gry na planszę.
if_text	text_renderer -> draw_mouse	vga_if	Obraz po napisach i komunikatach.
if_mouse	draw_mouse -> wyjścia VGA	vga_if	Finalny obraz z kursorem, mapowany na RGB, HS i VS.
uart_rx	PMOD JA3 -> pmod_comm_controller	pmod_uart_rx	Jednokierunkowy odbiór pakietów UART z drugiej płytki.
uart_tx	pmod_comm_controller -> PMOD JA2	pmod_uart_tx	Jednokierunkowe nadawanie pakietów UART do drugiej płytki.

4.2. Rozprowadzenie sygnału zegara

Osoby odpowiedzialne: MM, KM

Źródłem zegara jest wejście clk 100 MHz z płytki Basys 3. W module top_basys3 instancja IP clk_wiz_0 generuje dwa używane zegary: clk100MHz oraz clk65MHz.

Zegar 65 MHz jest zegarem pikselowym dla VGA 1024 x 768 i główną domeną logiki gry, renderowania oraz komunikacji PMOD UART. Zegar 100 MHz jest używany przez kontroler myszy PS/2 oraz układ debounce resetu.

W pozostałych modułach używane są wyłącznie zegary przekazane z top_basys3. Reset jest synchronizowany osobno w domenach 65 MHz i 100 MHz przez reset_sync, a asynchroniczne wejścia z myszy, PMOD i przełącznika są objęte ograniczeniami CDC w pliku XDC.

5. Implementacja

5.1. Lista zignorowanych ostrzeżeń Vivado.

Identyfikator ostrzeżenia	Liczba wystąpień	Uzasadnienie
Brak w warning_summary.log	0	Skrypt tools/warning_summary.sh wykazał CLEAR :) dla syntezy i implementacji.
REQP-1840	1	Ostrzeżenie DRC dla wejścia sterującego blokiem RAMB18 w ROM ust. Projekt przechodzi DRC bez błędów i spełnia timing.
SYNTH-6	5	Ostrzeżenia metodologii dla bloków RAM bez wbudowanego rejestru wyjściowego. Timing po routingu jest spełniony.

5.2. Wykorzystanie zasobów

Wykorzystanie zasobów według raportu top_basys3_utilization_placed.rpt po implementacji:

Zasób	Użyto	Dostępne	Wykorzystanie
Slice LUTs	2639	20800	12.69%
Slice Registers	1518	41600	3.65%
Block RAM Tile	19.5	50	39.00%
DSPs	0	90	0.00%
Bonded IOB	22	106	20.75%
BUFGCTRL	3	32	9.38%
MMCME2_ADV	1	5	20.00%

5.3. Marginesy czasowe

Raport top_basys3_timing_summary_routed.rpt potwierdza spełnienie ograniczeń czasowych. Dla całego projektu WNS = 3.623 ns, TNS = 0.000 ns, WHS = 0.070 ns, THS = 0.000 ns. Dla zegara clk100MHz WNS = 3.623 ns, a dla clk65MHz WNS = 4.393 ns.

6. Konfiguracja sprzętu

W trybie multiplayer używane są dwie płytki Basys 3 połączone przez UART na złączu PMOD JA. Sygnał JA2 jednej płytki należy połączyć z JA3 drugiej płytki i odwrotnie; płytki muszą mieć wspólną masę.

Do każdej płytki podłączony jest osobny monitor VGA oraz osobna mysz PS/2. Każda płytka generuje lokalnie obraz 1024 x 768 i obsługuje własną mysz użytkownika.

Przełącznik SW[0] ustawia identyfikator gracza, dlatego na jednej płytce powinien być ustawiony na 0, a na drugiej na 1. Przycisk BTNC jest lokalnym resetem sprzętowym; ekranowy RESET GRY czyści logikę gry i wysyła do przeciwnika pakiet RESET_GAME.

7. Film.

Link do ściągnięcia filmu:

https://drive.google.com/file/d/14UAfCMvFrB9GYF0HgPTvU_2AFvtOcuV1/view?usp=sharing